

Contents

Learnable Help System — Implementation Plan

1. Purpose & Scope

2. Why RAG, not minimax / RL

3. High-Level Architecture

4. UX Spec — “Always available in the Guide”

5. Data Model

6. Retrieval Pipeline — Two Phases

7. LLM Hosting

8. The Corpus (User Documentation)

9. The Learnable Layers (in order of complexity)

10. API Surface

11. Sprint Breakdown

12. Testing Requirements

13. Open Decisions

14. Risks & Mitigations

15. References

1

1

2

3

4

5

6

7

8

8

9

10

10

10

10

Learnable Help System — Implementation Plan

Status: v1 Draft (for review) **Author:** Joe Pruskowski (with Claude) **Date:** 2026-05-07

1. Purpose & Scope

Replace the placeholder Search/Help slot in the Guide drawer with an **always-available, learnable help system**. Users type a question into the existing footer text box on `GuidePanel.jsx:160–174`; the system retrieves relevant content from a curated corpus, streams a grounded answer from a small LLM, and captures structured feedback so quality can be improved over time.

The LLM lives behind a thin internal proxy (`xo-llm-*`) so the model can be swapped without touching `helpService`. **For v1 the proxy calls Groq’s hosted API** (free tier covers expected v1 traffic, ~1s latency). Self-hosted execution is a one-Dockerfile swap whenever data residency or vendor risk pushes us off Groq.

What this doc is: the architecture, data model, UX, hosting plan, and sprint breakdown for v1.

What this doc is not: the user-facing handbook itself. The corpus content lives in `/doc/Help_Corpus/` (a separate authoring effort tracked alongside this plan, see §8).

Non-goals for v1

- Self-hosted model execution. v1 uses Groq behind the swappable proxy; self-hosting is preserved as a near-zero-cost migration path (see §7).
- Voice / audio input.
- Multi-turn dialog state beyond the current panel session.
- Personalization beyond the page/journey context already on hand.
- Fine-tuning the LLM. Pure RAG (retrieval grounds the answer); model weights stay generic.

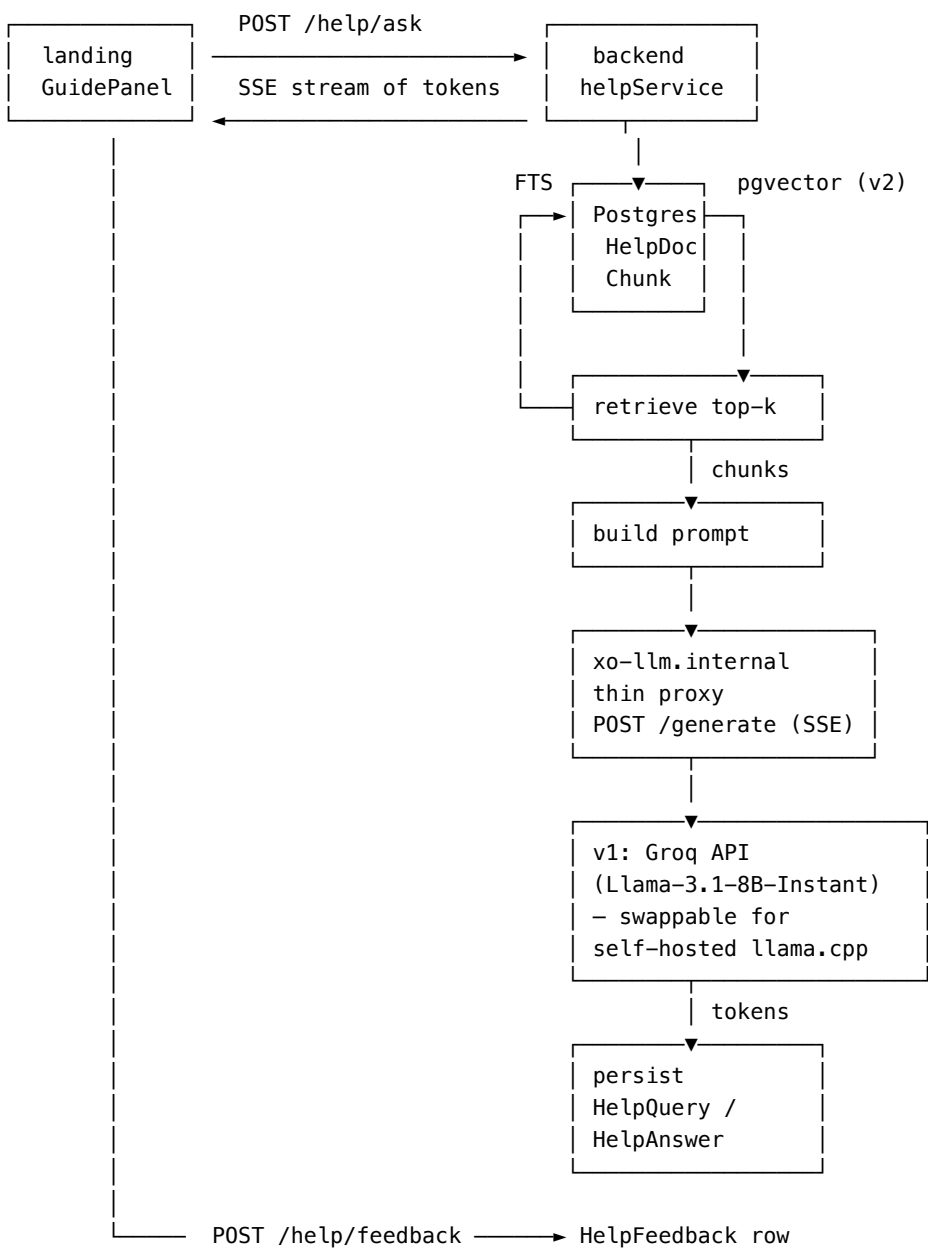
2. Why RAG, not minimax / RL

The minimax-with-RL analogy that motivated this work doesn’t transfer to question answering: there’s no discrete action space, no win/loss reward, no self-play. The mature pattern for in-app help is:

1. **Retrieval** — pull the most relevant chunks from a curated corpus
2. **Generation** — feed those chunks + the question to a small LLM that composes a grounded answer
3. **Feedback** — capture user signal (thumbs, categorical, implicit) on whether the answer was useful
4. **Improvement** — use the signal to (a) curate new docs where coverage is missing, (b) train a learned reranker over retrieval scores, and eventually (c) fine-tune the LLM on the best Q&A pairs

The “learnable” part is real but it’s data-driven, not RL: the reward signal is *user feedback*, the policy improvement is *better retrieval + better corpus + better answer style*, and the loop runs in an admin-curation cadence rather than online.

3. High-Level Architecture



Three deployment surfaces:

Surface	App	Notes
helpService	existing xo-backend-*	Owens retrieval, prompt build, persistence, SSE pass-through
LLM proxy	new xo-llm-* Fly app	Thin Bun/Node proxy, no DB. v1 forwards to Groq API; v2+ swappable to in-container llama.cpp
Vector store	existing xo-db-* Postgres	Add vector extension when v2 lands; FTS works on stock Postgres for v1

Why a separate proxy service: (1) `GRQ_API_KEY` (or any future provider key) lives only on `xo-llm-*` — never in the backend env, (2) swapping providers or moving to a self-hosted model is a one-Dockerfile change with no backend touches, (3) caching / retry / fallback logic lives in one place. `BACKEND → xo-llm-prod.internal:8080` mirrors how backend talks to `xo-tournament-prod`.

4. UX Spec — “Always available in the Guide”

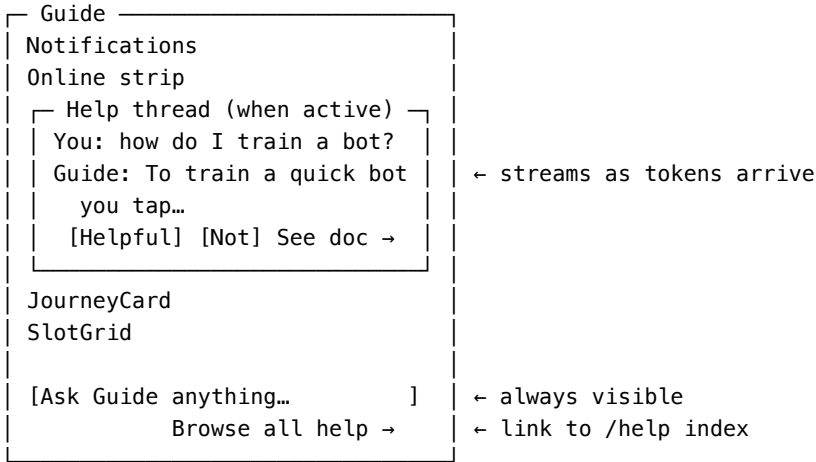
4.1 Anchor

The footer chat input on `GuidePanel.jsx:160–174` (currently a placeholder div) becomes a real input. Placeholder text remains “Ask Guide anything...”. Pressing Enter submits; Shift+Enter inserts a newline.

Below the input, a small text link “**Browse all help →**” routes to `/help` — a category-grouped index of all published `HelpDocs`. Provides a reading-first path for users who don’t yet know what to ask.

4.2 Inline thread, not modal

When a question is submitted, a `HelpThread` block appears in the panel’s scroll body **above** `JourneyCard` and `SlotGrid`. The drawer’s natural scroll keeps slots accessible.



Multiple turns stack within the thread. Closing the panel preserves the thread in `sessionStorage`; a new browser session starts empty (help context goes stale fast).

4.3 Response states

- **Pending** — Thinking... with a small spinner, replaced as soon as the first token arrives
- **Streaming** — tokens appended in real time
- **Complete** — feedback row revealed (Helpful / Not Helpful / “See full doc →”)
- **Error** — single-line “Couldn’t reach the Guide service. Try again?”

4.4 Feedback UX (graded friction)

Layer	Friction	Captured
Helpful / Not Helpful thumbs Categorical (if Not Helpful)	one tap one extra tap	HELPFUL / NOT_HELPFUL category ∈ {OFF_TOPIC, OUTDATED, WRONG, INCOMPLETE}
Optional comment	free text	comment
Implicit signal	none — logged automatically	follow-up within 60s, doc-link clickthrough

Signal locks after submission (“Thanks — that helps us improve.”). The answer stays visible.

4.5 Context auto-attached

Every `POST /help/ask` includes `{ route, journeyStep, currentSlot }` derived client-side. This goes to `HelpQuery.context` (jsonb) and the prompt template uses it for personalization (“the user is on step 3 of the Hook phase”). Free personalization with no extra UI.

4.6 Guests and rate limits

- **v1 is authed-only.** Guests see “Sign in to ask Guide questions” below the input. This anchors rate limiting on `userId` and avoids LLM-cost abuse vectors.
 - **Rate limit:** 50 questions/day per user, 5/min burst. Tracked via existing `resourceCounters` shape. Hitting the limit returns a friendly inline message, not an error. Cap is for abuse control, not cost control — Groq’s free tier easily absorbs all v1 traffic.
-

5. Data Model

All tables live in `packages/db/prisma/schema.prisma`. Migrations follow the standard `docker compose run --rm backend npx prisma migrate deploy` flow.

5.1 Authoring

HelpDoc	the curated source content	
└ id	uuid pk	
└ slug	text unique	stable URL/anchor (/help/<slug>)
└ title	text	
└ body	text	markdown
└ category	text	'system' 'ai-training' 'tournaments' 'gameplay' 'account'
└ tags	text[]	
└ status	enum	DRAFT PUBLISHED ARCHIVED
└ authorId	uuid → User	
└ version	int	bumped on edit; chunk invalidation key
└ createdAt		
└ updatedAt		

5.2 Retrieval

HelpChunk	retrievable slices (one doc → many chunks)	
└ id	uuid pk	
└ docId	uuid → HelpDoc on cascade delete	
└ position	int	ordering within doc
└ content	text	~200–500 token slice
└ tsv	tsvector	generated column from content (v1 retrieval)
└ embedding	vector(384) NULL	pgvector, populated in v2
└ docVersion	int	matches HelpDoc.version
└ createdAt		

index gin on tsv

index ivfflat on embedding (added with v2 migration)

5.3 Query log (training-data goldmine)

HelpQuery		
└ id	uuid pk	
└ userId	uuid → User NULL	nullable to ease guest support later
└ text	text	
└ textTsv	tsvector	generated from text (for analytics dedup)
└ embedding	vector(384) NULL	populated in v2
└ retrievedChunkIds	uuid[]	chunks pulled for this query
└ retrievalScore	jsonb	{ chunkId: { fts: float, vector: float, hybrid: float } }
└ topAnswerId	uuid → HelpAnswer	
└ context	jsonb	{ route, journeyStep, currentSlot, sessionId }
└ promptTemplate	text	'help.v1' – versioned template id
└ modelVersion	text	'qwen2.5-1.5b-q4@2026-05-07'
└ latencyMs	int	
└ createdAt		

5.4 Generated answer

HelpAnswer		
└ id	uuid pk	
└ queryId	uuid → HelpQuery on	cascade delete
└ chunkIds	uuid[]	chunks composed into the answer
└ rendered	text	markdown actually shown
└ rank	int	1-indexed; v1 only renders rank 1
└ tokensIn	int	
└ tokensOut	int	
└ stopReason	text	'eos' 'maxlen' 'truncated' 'error'

5.5 Feedback

HelpFeedback		
└ id	uuid pk	
└ userId	uuid → User	
└ queryId	uuid → HelpQuery	
└ answerId	uuid → HelpAnswer	
└ signal	enum	HELPFUL NOT_HELPFUL
└ category	enum NULL	OFF_TOPIC OUTDATED WRONG INCOMPLETE (only when NOT_HELPFUL)
└ comment	text NULL	
└ implicit	jsonb	{ followUpWithin60s: bool, docLinkClicked: bool, ... }
└ createdAt		

unique (userId, queryId, answerId) one feedback row per user per shown answer

5.6 Future: reranker tracking (deferred)

HelpReranker (added when reranker training begins, mirrors BotSkill)

└ id, version, params, trainingDataCount, deployedAt, ...

6. Retrieval Pipeline — Two Phases

6.1 v1 — Postgres FTS only

- HelpChunk.tsv is a generated column: `to_tsvector('english', content)`
- Query embedding is **just the user's text**, weighted by `ts_rank_cd` over the GIN index
- Top-5 chunks are pulled; top-1 goes to the prompt's grounding section

No embedding model needed, no vector extension, no extra infra. Ships fast and is honestly fine for “how do I create a bot?”-shaped questions.

6.2 v2 — Hybrid (FTS + pgvector)

- Add embedding column to both HelpChunk and HelpQuery
- Populate via a sentence-transformers MiniLM-L6-v2 (~80MB, CPU-fast, runs in `xo-llm`)
- Hybrid score: $0.4 * \text{normalize}(\text{fts_rank}) + 0.6 * \text{vector_similarity}$ as a starting weight
- Re-rank the top-20 of either source into a final top-5

v2 ships behind a SystemConfig flag so we can A/B and roll back.

6.3 v3 — Learned reranker (deferred)

Once HelpFeedback accumulates ~1k+ rows of (query, chunk, signal) triples, train a small reranker (LightGBM or a tiny dual-encoder) that re-orders the v2 top-20 → top-3. Deploy as a HelpReranker row + a flag in SystemConfig. Skip until data justifies it.

7. LLM Hosting

7.1 v1 — Groq behind a thin proxy

```
xo-llm-prod / xo-llm-staging      new Fly app, one shared-cpu machine each
├ Image: bun + minimal proxy (~50 lines)
├ Endpoints
│ ├── POST /generate    SSE stream – { prompt, maxTokens, stop[] } → tokens
│ ├── POST /embed       sync      – { texts[] } → { embeddings[] }   (v2)
│ └ GET  /health
├ Env
│ ├── GROQ_API_KEY      (only this service holds it; backend never sees it)
│ ├── INTERNAL_SECRET   (shared with backend – auth between services)
│ └ LLM_MODEL           'llama-3.1-8b-instant' versioned, logged into HelpQuery.modelVersion
└ No DB
```

The proxy receives a prompt over POST /generate, forwards it to Groq’s chat-completions endpoint, and re-streams Groq’s tokens back over SSE in the same shape. ~50 lines of glue code; no model logic, no GPU, no GGUF file.

7.2 Model choice — Llama-3.1-8B-Instant via Groq

Aspect	Value
Model	Llama-3.1-8B-Instant (Meta, hosted by Groq)
Latency, 200 tok	~1s end-to-end including network
Quality on grounded Q&A	Very good — handles RAG context cleanly, follows “answer only from source” instruction
Cost (v1 traffic, ~500 q/day)	\$0 (well under Groq’s 14,400 req/day free tier)
Cost ceiling (paid tier)	~\$0.05–\$0.10 per million tokens — pennies/month even at 10× growth

If quality ever falls short, Groq also hosts Llama-3.3-70B-Versatile (excellent quality, ~2s latency, still in free tier at low volume). Swap is one env var change.

7.3 Hardware

- **shared-cpu-1x** Fly machine (1 vCPU, 256 MB RAM) — ~\$2/mo per environment. Proxy does no inference; CPU and RAM are trivial.
- One machine each for staging + prod = ~\$4/mo total.
- No GPU. Model runs on Groq’s LPU infrastructure.

7.4 Self-hosting swap path (deferred)

The proxy boundary is what makes this reversible. When/if data-residency, vendor risk, or scale economics push us off Groq, the swap is:

1. Replace xo-llm Dockerfile to bundle node-llama-cpp + a GGUF model (e.g. Qwen2.5-1.5B Q4_K_M)
2. Re-implement /generate to call llama.cpp instead of Groq
3. Bump the Fly machine to performance-2x (4 vCPU, 8 GB) — ~\$30/mo per env
4. helpService, prompt template, schema, and feedback loop are all unchanged

Keep the original self-host plan handy:

Self-host model	Size (Q4_K_M)	Latency on perf-2x	Monthly cost (both envs)
Llama-3.2-1B-Instruct	~0.8 GB	~6–10s	~\$44
Qwen2.5-1.5B-Instruct	~1.0 GB	~10–15s	~\$60
Phi-3.5-mini (3.8B)	~2.4 GB	~20–35s	~\$120

7.5 Prompt template (v1)

You are the AI Arena Guide. You answer questions about the platform using only

the source material provided. If the answer is not in the source, say so plainly.
Keep responses under 200 words. Use Markdown.

User context:

- Route: {route}
- Journey step: {journeyStep}
- Current panel slot: {currentSlot}

Source material:

{topChunks joined with "----"}

Question:

{userQuestion}

Answer:

Versioned as help.v1. Future template revisions are stored on HelpQuery.promptTemplate so old rows remain replayable.

7.6 Cost guardrails

- Hard maxTokens: 400 per response
 - Trip the rate limiter (5/min, 50/day per user) at the helpService layer, *before* hitting xo-llm
 - LLM proxy caps concurrent requests (env var, e.g. 8 simultaneous); excess returns 429
 - On Groq paid tier ever, an absolute monthly token budget tripped by Fly metric → 429 with a friendly inline message
-

8. The Corpus (User Documentation)

The corpus is **the** quality lever. The model is generic; what makes the help good is the docs.

8.1 Authoring location

```
/doc/Help_Corpus/  
├ getting-started.md  
├ playing-tic-tac-toe.md  
├ bots-overview.md  
├ quick-bots.md  
├ ai-training-overview.md  
├ ai-training-q-learning.md  
├ ai-training-dqn.md  
├ ai-training-alphazero.md  
├ tournaments.md  
├ tables-and-spectating.md  
├ credits-tc-hpc-bpc.md  
├ profile-and-settings.md  
└ admin-features.md (gated; only surfaces for admins)
```

Each .md file is one authored unit. A seed:help npm script or um help-reindex command:

1. Parses each file (YAML frontmatter + body)
2. Upserts a HelpDoc row (slug from filename)
3. Chunks the body and replaces the doc's HelpChunk rows
4. Bumps HelpDoc.version

Doc edits flow through git like any other content; production reindex happens on deploy.

8.2 Frontmatter

```
----  
slug: ai-training-q-learning  
title: Q-Learning Bots  
category: ai-training
```

```
tags: [training, q-learning, ml]
status: PUBLISHED
admin_only: false
---
```

8.3 Chunking strategy

- ~300 tokens per chunk, with ~50 token overlap
- Split on Markdown headers first, then on sentence boundaries
- Each chunk preserves its parent heading in a prepended `## Section\n` line so retrieval surface still has structural context

8.4 Q&A pairs (deferred)

A separate `/doc/Help_Corpus/qa-pairs.jsonl` (or DB table) holds high-quality (question, ideal answer) pairs derived from real `HelpQuery` rows after launch. **Not needed for v1.** It's the substrate for v3 LoRA fine-tuning if/when we go there.

9. The Learnable Layers (in order of complexity)

Layer	Build effort	When
Curation queue — admin reviews NOT_HELPFUL queries, writes new docs	low (1 admin page over <code>HelpQuery</code> filtered by signal)	v1
Implicit signal collection — log follow-up timing + doc clickthrough	trivial	v1
Hybrid retrieval (FTS + vector)	medium (pgvector setup, embed pipeline)	v2, post-launch
Learned reranker	medium (training pipeline, deploy hook)	v3, after ~1k feedback rows
LoRA fine-tuning of the LLM on Q&A pairs	high (training infra + model swap pipeline)	v4, optional

The *real* learning loop in v1 is **curation**: real questions reveal where the corpus has gaps; admins fill them; retrieval improves immediately. This is the highest-leverage cycle and it ships first.

10. API Surface

10.1 User-facing

```
POST /api/v1/help/ask      { question, context }      SSE stream
POST /api/v1/help/feedback { queryId, answerId, signal, category?, comment? }
GET  /api/v1/help/docs/:slug single doc by slug, published only
GET  /api/v1/help/docs     published list, grouped by category – backs the /help index page
```

Frontend routes (landing):

```
/help      index page – categories + doc titles, public, no auth required
/help/:slug single doc page – public, renders HelpDoc.body as Markdown
```

10.2 Admin-facing (role gated)

```
GET  /api/v1/admin/help/queries      ?signal=NOT_HELPFUL      curation queue
GET  /api/v1/admin/help/queries/:id   single query + answer + feedback
POST /api/v1/admin/help/docs          CRUD: create
PUT  /api/v1/admin/help/docs/:id      CRUD: update (bumps version, reindexes chunks)
DELETE /api/v1/admin/help/docs/:id    soft-delete via status=ARCHIVED
POST /api/v1/admin/help/reindex       rebuild chunks for one or all docs
GET  /api/v1/admin/help/metrics       rollups: questions/day, helpful%, top categories
```

10.3 LLM service (internal only)

```
POST http://xo-llm-prod.internal:8080/generate  SSE { prompt, maxTokens, stop[] }
```



```
POST http://xo-llm-prod.internal:8080/embed { texts[] } v2
GET http://xo-llm-prod.internal:8080/health
```

Auth: shared INTERNAL_SECRET env var, mirrors how backend authenticates to tournament service.

11. Sprint Breakdown

Sprint 1 — Schema + corpus authoring + retrieval

- Prisma schema for HelpDoc, HelpChunk, HelpQuery, HelpAnswer, HelpFeedback
- Migration applied to dev / staging
- seed:help script + /doc/Help_Corpus/ with first 6–8 docs (getting started, bots, quick bots, tournaments, credits, gameplay)
- helpService.search(text) — FTS retrieval, returns top-5 chunks with scores
- Admin reindex endpoint + um help-reindex
- Backend tests: schema integrity, search relevance for hand-curated queries

Sprint 2 — LLM proxy + ask endpoint

- New Fly app xo-llm-staging — bun + ~50-line proxy that forwards to Groq’s chat-completions API and re-streams tokens as SSE
- GROQ_API_KEY + INTERNAL_SECRET set as Fly secrets on the new app only
- /generate SSE endpoint + /health
- helpService.ask(question, context) — retrieve → build prompt → stream from xo-llm → persist HelpQuery + HelpAnswer
- POST /api/v1/help/ask exposes the stream to the client
- Rate limiter (50/day, 5/min) + 401 for guests
- Backend tests: prompt template rendering, query persistence, rate-limit enforcement
- Proxy tests: SSE pass-through against a Groq mock; auth rejection without INTERNAL_SECRET

Sprint 3 — Guide UI + feedback + browse pages

- HelpInput.jsx replaces the GuidePanel footer placeholder
- HelpThread.jsx + HelpAnswer.jsx render streamed responses
- HelpFeedback.jsx — thumbs, categorical chips, optional comment
- helpStore.js (zustand) — current thread, sessionStorage persistence, send/feedback actions
- POST /api/v1/help/feedback + persistence
- “Browse all help →” link below the chat input
- HelpIndexPage.jsx at /help — category-grouped list of published docs, public
- HelpDocPage.jsx at /help/:slug — Markdown render of one doc, public
- Implicit-signal logging (follow-up timer, doc-link clickthrough)
- Component tests (vitest), E2E happy path (e2e/tests/help-basic.spec.js) covering both ask flow and browse flow

Sprint 4 — Admin curation + corpus expansion

- AdminHelpPage.jsx — list HelpDocs, edit, NOT_HELPFUL queue with one-click “create doc from this query”
- /api/v1/admin/help/* endpoints
- Corpus expanded to ~15 docs covering AI training in depth (Q-learning, DQN, AlphaZero), admin features
- Metrics rollout endpoint + small dashboard tile

Sprint 5+ (post-launch, after data) — v2 hybrid retrieval

- pgvector extension + migration
- Embedding pipeline (sentence-transformers MiniLM in xo-llm)
- /embed endpoint + reindex pass
- Hybrid search behind SystemConfig flag, A/B with FTS-only

Future (deferred)

- Reranker training (v3)
- LoRA fine-tuning (v4)
- Voice input
- Cross-session help history view in profile

12. Testing Requirements

Per project conventions (CLAUDE.md, feedback memory tests_before_completion):

- **Backend unit/integration:** every new endpoint, every helpService branch, rate limiter, prompt template renderer, reindex pipeline
 - **Component tests:** HelpInput, HelpAnswer, HelpFeedback, HelpThread — render, interaction, streamed-state transitions
 - **E2E:** e2e/tests/help-basic.spec.js — sign in, open Guide, ask question, see streamed answer, click Helpful, verify HelpFeed-back row exists
 - **LLM service:** smoke /health + a fixture-prompt /generate test (mock the model with a tiny GGUF for CI speed)
-

13. Open Decisions

Resolved: - **DECIDED** — **Authed-only for v1.** Guests see “Sign in to ask Guide questions”; static /help/:slug doc pages remain public. (§4.6) - **DECIDED** — **v1 model: Llama-3.1-8B-Instant via Groq** behind the swappable xo-llm proxy. Self-hosted Qwen2.5-1.5B is the documented swap path. (§7) - **DECIDED** — **Rate limit: 50 questions/day, 5/min burst** per user. Abuse control, not cost control. (§4.6) - **DECIDED** — **No cross-session help history view in v1.** Thread persists in sessionStorage only; new browser session starts empty. HelpQuery rows are permanent for training; UI surfacing of past Q&A deferred. (§4.2) - **DECIDED** — **Corpus authoring at /doc/Help_Corpus/.** Same git/PR workflow as design docs. Pandoc-PDF companion convention does NOT apply — these are user-facing pages consumed via in-app help and /help/:slug, not printables. (§8.1) - **DECIDED** — **“Browse all help →” affordance** below the chat input. Opens a category-grouped index of published HelpDocs at /help. Helps discoverability for new users; doesn’t compete with chat. (§4.1, §4.2)

All decisions resolved. Ready for Sprint 1 kickoff.

14. Risks & Mitigations

Risk	Mitigation
LLM latency tanks UX	Groq is ~1s end-to-end; stream tokens; show “Thinking...” instantly until first token
LLM hallucinates info not in the corpus	Prompt instructs “answer only from source”; feedback signal surfaces hallucinations for curation
Groq outage or rate-limit hits us	Proxy returns 503 with friendly inline message; no backend impact. Document escape hatch: Cloudflare Workers AI as an alternate provider, behind same proxy interface
Groq changes pricing or deprecates a model	xo-llm proxy boundary makes provider swap a Dockerfile change. Self-hosted Llama-3.2-1B is the always-available fallback (§7.4)
Sensitive user data leaving infra to Groq	Document this in privacy notes; questions logged in HelpQuery are reviewed periodically; if regulated data ever surfaces, swap to self-hosted using §7.4
Rate-limit abuse via shared accounts	Authed-only + per-user limit; consider per-IP limit later if needed
Postgres FTS quality is too weak	Hybrid with pgvector is the plan B and the schema already supports it
Corpus rot	HelpDoc.updatedAt surfaces stale docs in admin UI; OUTDATED feedback category routes them to the curation queue
Costs grow with traffic	Free tier covers expected v1; paid tier is per-token and pennies/month at 10× growth; hard maxTokens caps any single request

15. References

- landing/src/components/guide/GuidePanel.jsx — anchor for the input
- doc/Intelligent_Guide_Implementation_Plan.md — Guide drawer architecture

- `doc/Platform_Implementation_Plan.md` — broader platform roadmap
- `doc/ML_Training_Architecture.md` — distinction between this RAG help system and real bot ML training (the latter modifies model weights via Q-learning / DQN / AlphaZero; the former does not)